Risk minimization

True Risk: $R(f) = E(L(y, f(x))) = \int L(y, f(x))\, dP_{xy}$

Emp. Risk: $R_{emp}(f) = \frac{1}{n} \sum_{i=1}^{n} L(y^{(i)}, f(x^{(i)}))$

Optimal constant model: $f_c^* = \arg\min_c E_{xy}(L(y,c))$

Bayes optimal model: $f^*_{\mathcal{H}_{all}} = \arg\min_{f \in \mathcal{H}_{all}} E_{y|x}(L(y, f(x)))$

↳ gives Bayes Risk R^*

↳ = optimal point-wise model: $R(f) = E_x[E(L(y,f)|x)]$

Bayes regret

$R(\hat{f}_{\mathcal{H}}) - R^* = \underbrace{[R(\hat{f}_{\mathcal{H}}) - \inf_{f \in \mathcal{H}} (R(f))]}_{\text{estimation Err}} + \underbrace{[\inf_{f \in \mathcal{H}} (R(f)) - R^*]}_{\text{approx. Err}}$

Loss functions

Regression: residuals $r(x) = y - f(x)$

Classification: margin $v(x) = y \cdot f(x)$, $y \in \{-1, 1\}$

- symmetric: $L(y, f(x)) = L(f(x), y)$
- translational invariant: $L(y+a, f(x)+a) = L(y, f(x))$

pseudo residuals: $\tilde{r}(x) = -\frac{d}{df} L(y, f(x))$ Gradient

L2-Loss: $(y - f(x))^2$

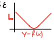
- pseudo residuals are normal residuals
- Result is $E(y|x)$ for theoretical Risk and mean of y for emp. Risk

L1-Loss: $|y - f(x)|$

- not differentiable for $y = f(x)$
- Result is median (y)

Huber-Loss

$L(y, f) = \begin{cases} \frac{1}{2}(y - f(x))^2 & , |y - f(x)| \le \varepsilon \\ \varepsilon |y - f(x)| - \frac{1}{2}\varepsilon^2 & , \text{else} \end{cases}$



- diff. & robust

ε-insensitive Loss

$L(y, f) = \begin{cases} 0 & , |y - f(x)| \le \varepsilon \\ |y - f(x)| - \varepsilon & , \text{else} \end{cases}$

Quantile Loss

$L(y, f) = \begin{cases} (1-\alpha)(y - f(x)) & , y < f(x) \\ \alpha (y - f(x)) & , y \ge f(x) \end{cases}$

- Risk minimizer is α-Quantile

0-1-Loss $L(y, f(x)) = \mathbb{1}_{\{y \ne f(x)\}}$

- for $y \in \{-1, 1\}$: $L(y, f(x)) = \mathbb{1}_{\{y \cdot f(x) < 0\}}$
- $R(f) = E_x\left(\sum_{k \in y} L(k, f(x)) \cdot P(y = k|x)\right)$
- optimal constant model is most probable class
- Bayes opt. model: $f^*(x) = \arg\max_k P(y = k | x = x_0)$

↳ $R^* = 1 - E_x(\max_k P(y = k|x))$

↳ min. Probability of being wrong

Bernoulli-Loss / Log-Loss

- negative log Likelihood of Bernoulli RV:

$L(y, \pi(x)) = -y \log(\pi(x)) - (1-y) \cdot \log(1 - \pi(x))$ for $\pi(x) \in [0,1]$

- hard penalty for confident wrong preds
- with margin: $L(y, f) = \log(1 + \exp(-y \cdot f(x)))$
- Risk minimizer $\pi^*(x) = P(y = 1 | x = x_0)$

↳ proof: derive loss wrt f with fixed x und setze $\pi(x) = c$ und $y = P(y=1|x=x)$

- optimal const. model: $\pi^c = \frac{1}{n} \sum_{i=1}^{n} y^{(i)}$, $y \in \{0, 1\}$

Proper Scoring Rule $E_{y \sim Q}(L(Q, P))$

- SR compares predictive to label distribution
- proper if $S(Q, Q) \le S(Q, P)$ $\forall P, Q$

For $Y \sim B(1, p)$

$\Rightarrow E_y(L(y, \pi)) = p \cdot L(1, \pi) + (1-p) \cdot L(0, \pi)$

↳ L1-Loss not proper

$E_y(L(y, \pi)) = p + \pi(1 - 2p) \Rightarrow \pi^* = \begin{cases} 0 & , p < 0.5 \\ 1 & , p > 0.5 \end{cases}$

↳ 0-1-Loss proper:

with hard labler $h(x)$ $E_y(L(y, h(x))) = \begin{cases} p & , h(x) = 0 \\ 1 - p & , h(x) = 1 \end{cases}$

any value in $[0.5, 1]$ gives $h(x) = 1$

Brier score: L2-Loss for classification

- $L(y, \pi(x)) = (y - \pi(x))^2$
- Risk minimizer: $\pi^*(x) = P(y = 1 | x = x_0)$

⇒ strictly proper scoring rule

- opt. const. model: $\frac{1}{n} \sum_{i=1}^{n} y^{(i)}$ for $y \in \{0, 1\}$

Calibration

$\pi(x) \in [0,1]$ calibrated if $P(y = 1 | \pi(x) = p) = p$

Hinge Loss

- $L(y, f(x)) = \max\{0, 1 - y \cdot f(x)\}$
- encourages confident true predictions
- Squared Hinge is more outlier sensitive

Exponential Loss

- $L(y, f(x)) = \exp(-y f(x))$
- never exactly 0

Max. Likelihood and ERM

- just define negative Log-Like. as Loss to minimize

↳ Set y^2 in Likelihood to $(y - f(x))^2$

- $\varepsilon \sim N(0, \sigma^2)$: neg log Like $\hat{=}$ L2 loss
- $\varepsilon \sim \text{Laplace}(0, \sigma^2)$: $\frac{1}{2\sigma} \exp(-\frac{|\varepsilon|}{\sigma})$ ⇒ L1 Loss
- $\varepsilon \sim B(1, p)$: Bernoulli Loss

⇒ every distr. has a loss

not every loss has a distr.

Generalization Error

- Expected Loss on unseen test data
- can be splitted into:
  - Variance of ε
  - Variance of learner at x ↳ deviance from avg. model
  - squared bias of learner ↳ diff btw avg. & true mod.

Risk Decomposition

$R(\hat{f}_{\mathcal{H}}) - R(f^*_{\mathcal{H}_{all}}) = R(\hat{f}_{\mathcal{H}}) - R(\tilde{f}_{\mathcal{H}})$ optim. error

$+ R(\tilde{f}_{\mathcal{H}}) - R(f^*_{\mathcal{H}})$ estim. error

$+ R(f^*_{\mathcal{H}}) - R(f^*_{\mathcal{H}_{all}})$ approx. error

- $\hat{f}_{\mathcal{H}}$: local minimum that we found on train data
- $\tilde{f}_{\mathcal{H}}$: global minimum on train data
- $f^*_{\mathcal{H}}$: best model in Hypothesis space
- $f^*_{\mathcal{H}_{all}}$: best overall model

⇒ $\begin{cases} \text{bias: approx. err. + estim. bias} \\ \text{Variance: optim. err. + estim. Variance} \end{cases}$

Multiclass classification $\mathcal{Y} = \{1, \dots, g\}$

$R(f) = E_{xy}[L(y, f(x))] = E_x\left[\sum_{k \in y} L(k, f(x_0)) \cdot P(y = k | x = x_0)\right]$

$\hat{f} = \arg\min_{f \in \mathcal{H}} R_{emp}(f) = \arg\min_f \sum_{i=1}^{n} L(y^{(i)}, f(x^{(i)}))$

- $f(x) = (f_1(x), \dots, f_g(x))$ scores
- $\pi(x) = (\pi_1(x), \dots, \pi_g(x))$ probabilities
- $h(x) = k \in \mathcal{Y}$ hard labels

0-1-Loss: $L(y, h(x)) = \mathbb{1}_{\{y \ne h(x)\}}$

- Bayes optimal: $h^*(x) = \arg\max_{\ell \in \mathcal{Y}} P(y = \ell | x = x_0)$
- optimal constant model: $h^*_c(x) = \text{mode}(y^{(i)})$

Brier: $L(y, \pi(x)) = \sum_{k=1}^{g} (\mathbb{1}_{\{y = k\}} - \pi_k(x))^2$

- optimal constant model: π_c with obs. freq.

Log-Loss: $L(y, \pi(x)) = -\sum_{k=1}^{g} \mathbb{1}_{\{y = k\}} \cdot \log(\pi_k(x))$

Softmax Regression (Logistic Reg. for multiclass)

$f_k(x) = \theta_k^T x$, $k = 1, \dots, g$ gives confidence in class k

$\Rightarrow \pi_k(x) = s(f(x))_k = \frac{\exp(\theta_k^T x)}{\sum_{j=1}^{g} \exp(\theta_j^T x)}$; $\sum \pi_k = 1$

↳ Log-Loss

Surprisal: $-\log_2(p(x))$ ⇒ high surprisal $\hat{=}$ less likely

Entropy

discrete RV X: $H(X) = -E[\log_2(p(X))] = -\sum_{x \in \mathcal{X}} p(x) \cdot \log_2 p(x)$

↳ „how hard to predict the outcome"

⇒ discrete: Entropy maximal for uniform distr.

- $H(X) \ge 0$

Differential entropy

for continuous RV X (also for Random Vektor $\underline{X}$)

$h(X) = -E(\log(f(X))) = -\int_{\mathcal{X}} f(x) \log f(x)\, dx$

$U[0, a]$: $h(X) = \log(a)$

$N[\mu, \sigma^2]$: $h(X) = \log(\sigma \cdot \sqrt{2\pi}) + \frac{1}{2}$

- can be negative
- additive for independent RVs
- if restricted to variance, maximized by normal
- $h(X + a) = h(X)$
- $h(aX) = h(X) \cdot \log|a|$

Joint Entropy

discrete RVs X, Y: $H(X, Y) = -\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \cdot \log_2 p(x, y)$

$= -E[\log p(X, Y)]$

cont RVs X, Y: $h(X, Y) = -\int_{\mathcal{X} \times \mathcal{Y}} p(x, y) \cdot \log p(x, y)\, dx\, dy$

- for X, Y indep.: $H(X, Y) = H(X) + H(Y)$

↳ because then $p(x, y) = p(x) \cdot p(y)$

- $H(X, X) = H(X)$

Conditional Entropy

- „Remaining uncertainty of Y if X is given"

$H(Y|X) = -E[\log p(Y|X)] = -\sum_x \sum_y p(x, y) \cdot \log p(y|x)$

⇒ $H(X, Y) = H(X) + H(Y|X)$

- $H(X|X) = 0$
- $H(Y|X) \le H(Y)$ (proof mit mutual Info)

Maximum Entropy principle

- choose distribution with maximum entropy to not make any assumptions about the unknown

↳ already known info in form: $E[g_m(X)] = \alpha_m$

$\Rightarrow \frac{\delta}{\delta p(x)} L(p(x), \lambda_m)$

$= \frac{\delta}{\delta p(x)} -\sum_{x \in \mathcal{X}} p(x) \log p(x) + \lambda_0 \cdot \left(\sum_{x \in \mathcal{X}} p(x) - 1\right) + \sum_m \lambda_m \sum_{x \in \mathcal{X}} g_m(x) \cdot p(x) - \alpha_m) \overset{!}{=} 0$

Kullback-Leibler Divergence

- p and q are distr. with same support
- p: original distr.; q: approx. of p
- discrete: $D_{KL}(p \| q) = E_p\left(\log \frac{p(X)}{q(X)}\right) = \sum_{x \in \mathcal{X}} p(x) \cdot \log \frac{p(x)}{q(x)}$
- cont.: $D_{KL}(p \| q) = E_p\left(\log \frac{p(X)}{q(X)}\right) = \int_{x \in \mathcal{X}} p(x) \cdot \log \frac{p(x)}{q(x)}\, dx$
- „Expected Log-Likelihood Ratio of p and q"
- $0 \cdot \log(0) = 0$; $p \cdot \log\left(\frac{p}{0}\right) = \infty$
- $D_{KL}(p \| q) \ne D_{KL}(q \| p)$
- $D_{KL}(p \| q) \ge 0$; „=" for $p = q$

Cross-Entropy

„How hard to predict X if we encode p with q"

discrete: $H(p \| q) = -\sum_{x \in \mathcal{X}} p(x) \cdot \log(q(x)) = -E_p(\log(q(X)))$

- $H(p \| q) = H(p) + D_{KL}(p \| q)$

Cont.: $h(p \| q) = -\int_{x \in \mathcal{X}} p(x) \cdot \log(q(x))\, dx = -E_p(\log(q(X)))$

- $h(p \| q) = h(p) + D_{KL}(p \| q)$

Minimizing KL $\hat{=}$ Maximizing Likelihood

$\arg\min_\theta D_{KL}(p \| q_\theta) = \arg\min_\theta -E_{x \sim p}(\log q(x|\theta))$

$= \arg\max_\theta E_p(\log q(x|\theta))$

For data x: $= \arg\max_\theta \frac{1}{n} \sum_{i=1}^{n} \log q(x^{(i)}|\theta)$

Mutual Information

- Information known about X when knowing Y

⇒ D_{KL} between $p(x, y)$ and $p(x) \cdot p(y)$

$I(X; Y) = E_{p(x,y)}\left(\log \frac{p(X,Y)}{p(X) q(Y)}\right)$

$= \sum_x \sum_y p(x, y) \cdot \log \frac{p(x,y)}{p(x) q(y)}$ (or Integral for cont.)

- $I(X; Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$
- $I(X; Y) \ge 0$; „=" for X, Y independent

Source Coding

- assigning shorter codewords to more frequent words
- Prefix code: No codeword is a prefix of any other
- for random distribution of words p the optimal average codeword length is H(p)
- Encoding p with q gives suboptimal codeword length $H(p \| q)$ (Cross Entropy)

Curse of Dimensionality

- in high dimensions, most volume is in outer shell
- in high dimensions you have a lot of regions

↳ might not have enough data to discribe all regions

- for uniformly distr. data most observations will fall on boundry of high-dim. space
- the more dimensions, the closer are max and min of distance between points

What can we do?

1) Dimension reduction (PCA, Feature selection)
2) Collect more data
3) Regularization

↳ explicit: ridge / Lasso

↳ implicit: early stopping

Regularization

- explicit: penalize model complexity (L1, L2)
- implicit: early stopping, ensembling, dropout

Regularized Risk:

$R_{reg}(f) = R_{emp}(f) + \lambda \cdot J(f) = \sum_{i=1}^{n} L(y^{(i)}, f(x^{(i)})) + \lambda J(f)$

↳ $J(f)$: complexity penalty

↳ $\lambda \ge 0$: complexity control parameter

⇒ helps when unregularized design matrix isn't full rank

- normally exclude θ_0 from $J(\theta)$

⇒ infinitely regularized model is const., not 0

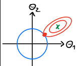
⇒ needs standardization: if downscale features, coeffs become larger → more reg.

Ridge Regularization

$\hat{\theta}_{ridge} = \arg\min_\theta \sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2 + \lambda \sum_{j=1}^{p} \theta_j^2$

$= \arg\min_\theta \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2$

$\Rightarrow = (X^T X + \lambda I)^{-1} X^T y$



$\hat{\theta}$: Risk minimizer without regularization: $\arg\min R_{emp}$

θ: $\arg\min R_{emp}$ for higher λ values

θ: $\|\theta\|_2^2$ squared length of θ

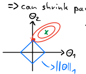
Lasso Regularization

$\hat{\theta}_{Lasso} = \arg\min_\theta \sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2 + \lambda \sum_{j=1}^{p} |\theta_j|$

$= \arg\min_\theta \|y - X\theta\|_2^2 + \lambda \|\theta\|_1$

⇒ not differentiable

⇒ can shrink parameter values to 0



- if two features are highly correlated, Lasso excludes one of them

Elastic Net

$R_{elnet}(\theta) = \sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2$

$= \sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2 + \lambda((1-\alpha)\|\theta\|_1 + \alpha \|\theta\|_2^2)$

↳ λ controls regularization

α controls proportion of L2

L0 Regularization

$R_{L0} = R_{emp}(\theta) + \lambda \cdot \sum |\theta_j|^0$

↳ penalizes number of features

↳ example: AIC, BIC

MCP (Minimax Concave Penalty)

$MCP(\theta | \lambda, \gamma) = \begin{cases} \lambda|\theta| - \frac{\theta^2}{2\gamma} & , |\theta| \le \gamma\lambda \\ \frac{1}{2}\gamma \cdot \lambda^2 & , |\theta| > \gamma\lambda \end{cases}$

⇒ less bias than L1/L2 (const in θ)

Structural Risk Minimization

- with increasing hypothesis space size $\mathcal{H}_0 < \mathcal{H}_1 < \dots$ chose smallest $\mathcal{H}_m$ that isn't significantly outperformed

Bayesian Priors

- parameterized distr. $p(y|\theta, x)$; prior $p(\theta)$

$\Rightarrow p(\theta | x, y) \propto p(y|\theta, x)\, q(\theta)$

↳ to maximize, minimize $\underbrace{-\log p(y|\theta, x)}_{\hat{=} L(y, f(x|\theta))} \underbrace{- \log q(\theta)}_{J(\theta)}$

- Gaussian prior ⇒ L2
- Laplace prior ⇒ L1
- flat prior ⇒ no regularization

L2 as weight decay

$\frac{\delta}{\delta\theta} R_{reg} = (\nabla_\theta R_{emp}) + \lambda\theta$

$\Rightarrow \theta^{(new)} = \theta^{(old)} - \alpha(\nabla_\theta R_{emp}(\theta^{(old)}) + \lambda\theta^{(old)})$

$= \underbrace{\theta^{(old)}(1 - \alpha\lambda)}_{\text{shrinking}} - \underbrace{\alpha \nabla_\theta R_{emp}(\theta^{(old)})}_{\text{gradient descent}}$

Taylor approximation of L2 at minimizer $\hat{\theta}$

$\tilde{R}_{emp}(\theta) = R_{emp}(\hat{\theta}) + \frac{1}{2}(\theta - \hat{\theta})^T \cdot H \cdot (\theta - \hat{\theta})$

↳ because first order term at minimizer $\hat{\theta}$ is 0

$\Rightarrow \tilde{R}_{reg}(\theta) = \tilde{R}_{emp}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$

Find $\hat{\theta}_{ridge}$ in terms of $\hat{\theta}$:

$\nabla_\theta \tilde{R}_{reg}(\theta) = 0 \Rightarrow H(\theta - \hat{\theta}) + \lambda\theta = 0$

$\Rightarrow \hat{\theta}_{ridge} = (H + \lambda I)^{-1} H\hat{\theta}$ (H is 2. deriv.)

Linear SVM

Data: $(x^{(i)}, y^{(i)})$, $x \in \mathbb{R}^n$, $y \in \{-1, 1\}$

Hard margin linear SVM

- data separated by $f(x) = \theta^T x + \theta_0$
- $d(f, x^{(i)}) = y^{(i)} \cdot \frac{\theta^T x^{(i)} + \theta_0}{\|\theta\|}$ (sign tells if correctly classif.)
- Margin $\gamma = \min_i \{d(f, x^{(i)})\}$

⇒ we search for $\max_\theta \gamma$ s.t. $d(f, x^{(i)}) \ge \gamma$

To ensure uniqueness of hyperplan, we set $\|\theta\| = 1/\gamma$

⇒ Therefore: $\max \frac{1}{\|\theta\|}$ s.t. $y^{(i)}(\theta^T x^{(i)} + \theta_0) \ge 1$ $\forall i$

Primal Problem

$\Rightarrow \min_\theta \frac{1}{2}\|\theta\|_2^2$ s.t. $y^{(i)}(\theta^T x^{(i)} + \theta_0) \ge 1$ $\forall i$

- Support Vectors have dist $\gamma = \frac{1}{\|\theta\|}$ to f

Deriving Dual Problem

$L(\theta, \theta_0, \alpha) = \frac{1}{2}\|\theta\|^2 - \sum_{i=1}^{n} \alpha_i [y^{(i)}(\theta^T x^{(i)} + \theta_0) - 1]$

s.t. $\alpha_i \ge 0$ $\forall i$

Looking for stationary point of $L(\theta, \theta_0, \alpha)$ in θ, θ_0

⇒ gives $0 = \sum_{i=1}^{n} \alpha_i y^{(i)}$

↳ sum of class „+" α_i and class „−" α_i is equal

Dual problem:

$\max_{\alpha \in \mathbb{R}^n} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle$

s.t. $\sum_{i=1}^{n} \alpha_i y_i = 0$ $(\Rightarrow \sum_{y_i = „+"} \alpha_i = \sum_{y_i = „-"} \alpha_i)$

$\Rightarrow \hat{\theta} = \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)}$

- only for support vectors $\alpha_i \ne 0$ but not all support vectors have $\alpha_i = 0$

Soft Margin linear SVM Primal

- allow margin violation: $y^{(i)}(\theta^T x^{(i)} + \theta_0) \ge 1 - \xi^{(i)}$

$\Rightarrow \min_{\theta, \theta_0, \xi} \underbrace{\frac{1}{2}\|\theta\|^2}_{\text{inverted size of margin}} + C \sum_{i=1}^{n} \xi^{(i)}$ (minimize violations)

s.t. $y^{(i)}(\theta^T x^{(i)} + \theta_0) \ge 1 - \xi^{(i)}$ and $\xi^{(i)} \ge 0$

Soft Margin linear SVM Dual

$\max_{\alpha \in \mathbb{R}^n} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle$

s.t. $\sum_{i=1}^{n} \alpha_i y_i = 0$ and $0 \le \alpha_i \le C$

↳ the larger C, the more relevant are margin violations and the less relevant is size of the margin

- $\alpha_i = 0 \Rightarrow x^{(i)}$ non-SV, $\xi^{(i)} = 0$
- $0 < \alpha_i < C \Rightarrow x^{(i)}$ is SV on margin, $\xi^{(i)} = 0$
- $\alpha_i = C \Rightarrow x^{(i)}$ is SV with margin violation, $\xi^{(i)} > 0$

SVM as ERM with Hinge Loss and L2

- Primal problem can be rewritten:

$\min_{\theta, \theta_0} \underbrace{\frac{1}{2}\|\theta\|_2^2}_{\text{L2-Reg.}} + C \cdot \underbrace{\sum_{i=1}^{n} \max\{1 - y^{(i)} \cdot f(x^{(i)}), 0\}}_{\text{Hinge-Loss}}$

Non-linear SVM

- transformation of features to create new features

Kernel trick

$\max_\alpha \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \phi(x^{(i)}); \phi(x^{(j)}) \rangle$ ← replace it by $k(x^{(i)}, x^{(j)})$

s.t. $0 \le \alpha_i \le C$, $\sum_{i=1}^{n} \alpha_i y^{(i)} = 0$

Kernel: $k: \mathcal{X} \times \mathcal{X} \to \mathbb{R}$

with: $k(x, \tilde{x}) = k(\tilde{x}, x)$

- K with $K_{ij} = k(x^{(i)}, x^{(j)})$ is positive semi-definite

↳ $x^T K x \ge 0$ $\forall x \in \mathbb{R}^n$

- $\lambda \ge 0$: $\lambda \cdot k$ is a kernel
- $k_1 + k_2$ is a kernel
- $k_1 \cdot k_2$ is a kernel

Homogeneous Polynomial Kernel

$k(x, \tilde{x}) = (x^T \tilde{x})^d = (x_1 \tilde{x}_1 + \dots + x_p \tilde{x}_p)^d$

Non-homogeneous Polynomial Kernel

$k(x, \tilde{x}) = (x^T \tilde{x} + b)^d$

Mercer's Theorem

- for every kernel, there exists a hilbert space where the kernel acts as dot product

↳ reproducing property:

$\langle k(x, .), k(\tilde{x}, .) \rangle = \langle \phi(x), \phi(\tilde{x}) \rangle = k(x, \tilde{x})$

$\Rightarrow \phi(x) = k(x, .)$ is basis function

- Hilbert space for $k(x, .)$ and $\mathcal{X}$ is the span of $k(x, .)$ for all $x \in \mathcal{X}$

↳ Span of x: $\sum_{i=1}^{\infty} \alpha_i \cdot k(x_i, .)$; $\sum_{i=1}^{\infty} \alpha_i^2 < \infty$

⇒ inner product:

$\langle \sum_{i=1}^{\infty} \alpha_i k(x^{(i)}, .), \sum_{j=1}^{\infty} \beta_j k(x^{(j)}, .) \rangle$

$= \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} \alpha_i \cdot \beta_j \cdot \langle k(x^{(i)}, .); k(x^{(j)}, .) \rangle$

$= \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} \alpha_i \cdot \beta_j \cdot k(x^{(i)}, x^{(j)})$

Representer Theorem

Solution of SVM: $\theta = \sum_{i=1}^{n} \alpha_i \cdot \phi(x^{(i)})$; $\alpha_i \in \mathbb{R}$

Prediction for new x

$f(x) = \sum_{i=1}^{n} \alpha_i \cdot k(x^{(i)}, x) + \theta_0$

↳ $\alpha_i = 0$ for non-SV

Gaussian Kernel

$k(x, \tilde{x}) = \exp\left(-\frac{\|x - \tilde{x}\|^2}{2\sigma^2}\right)$

↳ creates gaussian bumps in Hyperplane

↳ wiggly parameter is σ^2

- SVMs very sensitive to choice of Kernel and choice of hyperparameters
- to get an intuition on hyper pars, compute quantiles of pairwise distances between $x^{(i)}$
- SVMs are non-parametric: we don't have to decide on number of parameters
- Risk of universally consistent SVMs (so with universal kernel) converges to Bayes Risk

Boosting – AdaBoost

- $y \in \{-1; 1\}$
- Base Learner $b^{[m]} = b(x, \theta^{[m]})$
- misspecified points of $b^{[m]}$ get more weight for $b^{[m+1]}$ training
- final model is weighted average: $\hat{f}(x) = \sum_{m=1}^{M} \hat{\beta}^{[m]} \cdot \hat{b}^{[m]}(x)$

↳ number of iters M is tuning parameter

- $\beta^{[m]}$: for each $b^{[m]}$ compute:

$err^{[m]} = \sum_{i=1}^{n} w^{[m](i)} \cdot \mathbb{1}_{\{y^{(i)} \ne b^{[m]}(x^{(i)})\}}$

$\Rightarrow \hat{\beta}^{[m]} = \frac{1}{2} \cdot \log\left(\frac{1 - err^{[m]}}{err^{[m]}}\right)$

- $w^{[m+1](i)} = w^{[m](i)} \cdot \exp(-\hat{\beta}^{[m]} \cdot \underbrace{y^{(i)} \cdot \hat{b}^{[m]}(x^{(i)})}_{\in \{-1; 1\}})$
- normalize $w^{[m+1]}$ → Sum = 1

$\Rightarrow \hat{f}(x) = \sum_{m=1}^{M} \hat{\beta}^{[m]} \cdot \hat{b}^{[m]}(x)$; $h(x) = \text{sign}(\hat{f}(x))$

Gradient Boosting

- Additive model: $f(x) = \sum_{m=1}^{M} \alpha^{[m]} b(x, \theta^{[m]})$

with Risk $R_{emp}(f) = \sum_{i=1}^{n} L(y^{(i)}, \sum_{m=1}^{M} \alpha^{[m]} \cdot b(x^{(i)}, \theta^{[m]}))$

↳ high-dimensional, so do one component at a time

Algorithm:

- initialize $\hat{f}^{[0]}(x)$ as optimal constant model
- do m = 1 to M:
  1) $(\hat{\alpha}^{[m]}, \hat{\theta}^{[m]}) = \arg\min_{\alpha, \theta} \sum_{i=1}^{n} L(y^{(i)}, \hat{f}^{[m-1]} + \alpha \cdot b(x^{(i)}, \theta))$
  2) $\hat{f}^{[m]}(x) = \hat{f}^{[m-1]}(x) + \hat{\alpha}^{[m]} \cdot b(x, \hat{\theta}^{[m]})$

Boosting as Gradient Descent

- to find $b(x, \theta^{[m]})$, go in direct of neg. gradient

$\hat{f}^{[m]}(x) = \hat{f}^{[m-1]}(x) - \alpha \cdot \underbrace{\frac{\delta L(y^{(i)}, f(x^{(i)}))}{\delta f^{[m-1]}(x^{(i)})}}_{\text{pseudo-residuals } \tilde{r}^{[m]}}$

↳ L2: $\tilde{r}(f) = y - f(x)$; L1: $\tilde{r}(f) = \text{sign}(y - f(x))$

⇒ this would be pointless because we just memorize the training data

to find $\theta^{[m]}$

⇒ therefore we try to find the parameterization of the base learner b, that brings b closest to the pseudo residuals

$\Rightarrow b(x, \hat{\theta}^{[m]})$, $\hat{\theta}^{[m]} = \arg\min_\theta \sum_{i=1}^{n} [\tilde{r}^{[m](i)} - b(x^{(i)}, \theta)]^2$

↳ base learner fitted to residuals

to find $\alpha^{[m]}$

$\hat{\alpha}^{[m]} = \arg\min_\alpha \sum_{i=1}^{n} L(y^{(i)}, \hat{f}^{[m-1]}(x) + \alpha \cdot b(x, \hat{\theta}^{[m]}))$

Regularization in Gradient boosting

Stochastic gradient descent

- each iteration base learner is fitted on subsample of pseudo residuals

For boosted trees 3 hyperparameters:

- Number of base Learners M
- Learning rate α
- maximal tree depth

GB for Classification $\mathcal{Y} = \{0, 1\}$

Bern.-Loss: $L(y, f(x)) = -y \cdot f(x) + \log(1 + \exp(f(x)))$

$\Rightarrow \tilde{r}(f) = y - \frac{\exp(f(x))}{1 + \exp(f(x))}$

$\mathcal{Y} = \{1, \dots, g\}$ multiclass

$\pi_k(x) = \exp(f_k(x)) / \sum_j \exp(f_j(x))$

Multin. Loss: $L(y, f_1(x), \dots, f_g(x)) = -\sum_{k=1}^{g} \mathbb{1}_{\{y = k\}} \cdot \ln(\pi_k(x))$

$\Rightarrow \tilde{r}(f_k(x)) = \mathbb{1}_{\{y = k\}} - \pi_k(x)$

Terminal Coefficients

$T_{tree}: b(x) = \sum_{l=1}^{T} c_l \cdot \mathbb{1}_{\{x \in R_l\}}$

↳ R_l: terminal region; c_l: terminal constant

⇒ GB: $f^{[m]}(x) = f^{[m-1]}(x) + \nu \sum_{l=1}^{T} c_l^{[m]} \cdot \mathbb{1}_{\{x \in R_l^{[m]}\}}$

XGBoost

- 3 active regularization terms:

$R_{reg}^{[m]} = \sum_{i=1}^{n} L(y^{(i)}, f^{[m-1]}(x^{(i)}) + b^{[m]}(x^{(i)}))$
$+ \lambda_1 J_1(b^{[m]}) + \lambda_2 \frac{1}{2} J_2(b^{[m]}) + \lambda_3 \frac{1}{2} J_3(b^{[m]})$

↳ $J_1(b^{[m]})$: Number of leaves

↳ $J_2(b^{[m]})$: L2 penalty for leave values

↳ $J_3(b^{[m]})$: L1 penalty for leave values

- uses stochastic GB:
  - random subset of features used for splits
- when computing $f^{[m]}$ we use $f^{[m-1]}$ but drop random $b^{[j]}$ in $f^{[m-1]}$
  - ↳ if you drop none, it's normal GB
  - ↳ if you drop all $b^{[j]}$, it's a Random Forest

Componentwise GB

- aim: strong & interpretable model
  - automatic component selection
- use multiple different kinds of base learners
  - ↳ but use additive base learners, so that for two base learners of same type the parameters can just be added
  - $b_1(x, \theta^{[j]}) + b_2(x, \theta^{[j]}) = b_3(x, \theta^{[j]} + \theta^{[j]})$
- in each step multiple base learners are fitted to residuals and only best $b^{[j]}$ is added to the model
- usually each base learner for only one feature
- handling of categorical x_j with G classes
  - ↳ simultaneously: $b_j(x_j | \theta_j) = \sum_{g=1}^{G} \theta_{j,g} \cdot \mathbb{1}_{\{x_j = g\}}$
    - ↳ $\hat{\theta}_{j,g} = \frac{1}{n_g} \cdot \sum_{i=1}^{n} y^{(i)} \cdot \mathbb{1}_{\{x_j^{(i)} = g\}}$
  - ↳ individually: $b_{j,g}(x_j, \theta_{j,g}) = \theta_{j,g} \cdot \mathbb{1}_{\{x_j = g\}}$
    - ↳ non informative categories not included
    - ↳ $\hat{\theta}_{j,g} = \frac{1}{n_g} \cdot \sum_{i=1}^{n} y^{(i)} \cdot \mathbb{1}_{\{x_j^{(i)} = g\}}$
    - ↳ slower than simultaneous

Relation to GLM

- by specifying the negative log-likelihood of exp. fam. distribution, CWB is equivalent to GLM

Relation to GAM

- with p.o. b-splines as base learners, CBlw equivalent to GAM

Nonlinear effect decomposition

- Each base learner decomposed into constant, linear and nonlinear part

$\Rightarrow b_j(x_j, \theta^{[m]}) = b_{j,const}(\theta_{j,const}^{[m]}) + b_{j,lin}(\theta_{j,lin}^{[m]}) + b_{j,nonlin}(\theta_{j,nonlin}^{[m]})$

$= \theta_{j,const}^{[m]} + x_j \cdot \theta_{j,lin}^{[m]} + s(x_j, \theta_{j,nonlin}^{[m]})$

Bayesian LM

Likelihood: $y | X, \theta \sim \mathcal{N}(X^T \theta, \sigma^2 I)$

Prior: $\theta \sim \mathcal{N}(0, \tau^2 I)$

Posterior: $\theta | y, X \sim \mathcal{N}[\sigma^2 A^{-1} X^T y, \; A^{-1}]$

↳ $A = \sigma^2 X^T X + \frac{1}{\tau^2} I$

Gaussian Process

Any subset $\{x^{(1)}, \dots, x^{(n)}\} \subset \mathbb{R}^p$: $f \sim (f(x_1), \dots, f(x_n)) \sim \mathcal{N}(m, k)$

↳ $m \in \mathbb{R}^n$; $k \in \mathbb{R}^{n \times n}$

↳ $k_{ij} = k(x^{(i)}, x^{(j)})$

- 3 kinds:
  - stationary: $k(x^{(i)} - x^{(j)})$
  - iso-topic: $k(\|x^{(i)} - x^{(j)}\|)$
  - dot product: $k(x^{(i)T}(x^{(j)}))$

$corr = \frac{Cov}{\sqrt{Var_i \cdot Var_j}}$

Predicting with GP

- we know $x, f, k(\cdot,\cdot)$
- new data: $x_* \rightarrow f_* = f(x_*)$
- Posterior predictive:

$f_* | x_*, f, x \sim \mathcal{N}(k_*^T k^{-1} f, \; k_{**} - k_*^T K^{-1} k_*)$

↳ $k_{**} = k(x_*, x_*)$

↳ $k_* = (k(x_*, x^{(1)}), \dots, k(x_*, x^{(n)}))^T$

↳ $K = (k(x^{(i)}, x^{(j)}))_{ij}$

Losses:

- L2: Risk minimizer: $E_{y|x}(y|x) = f^*(x)$
  - Bayes Risk: $E_x(Var_{y|x}(y|x))$
  - opt const mod: $E_y(y)$
  - Risk of opt const mod: $Var_y(y)$
- 0/1: Risk minimizer: $\text{argmax}_k \; P(y=k|x)$
  - Bayes Risk: $1 - E_x(\max_k P(y=k|x))$
  - opt const mod:
  - Risk of opt const mod:
- Log: Risk minimizer: $P(y=1|x)$ — Entropy
  - Bayes Risk: $E_x(H_{y|x}(y|x))$
  - opt const mod: $P(y=1)$
  - Risk of opt const mod: $H_y(y)$
- Brier: Risk minimizer: $P(y=1|x)$
  - Bayes Risk: $E_x(Var_{y|x}(y|x))$
  - opt const mod: $P(y=1)$
  - Risk of opt const mod: $Var(y)$

Proof that k is kernel

⇒ Symmetric: $k(x, \tilde{x}) = k(\tilde{x}, x)$

⇒ positive definite: $a^T K a \geq 0$

⟹ or counter example

Pseudo Residuals

$\tilde{r}(f) = -\frac{\delta L(y,f)}{\delta f}$

Hard margin linear SVM

$\gamma = \min_i \{d(f, x^{(i)})\}$

$\|\theta\| = 1/\gamma$

Θ: - Gleichung $x_2 = a \cdot x_1 + b$ für f aufstellen und so umstellen, dass $=0$

- skalieren, sodass $\|\theta\| = 1/\gamma$

⇒ auf θ_0 aufpassen, wird auch skaliert

Kernel Trick

- für when data is not linearly separable, because you don't have to calculate the transformations $\phi(x)$ + features but can directly calculate the dot product

Noisy GP

$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$

$Cov(y^{(i)}, y^{(j)}) = k(x^{(i)}, x^{(j)}) + \sigma^2 \underbrace{\delta_{ij}}_{\begin{cases} 1 & , i=j \\ 0 & , \text{else} \end{cases}}$

$\Rightarrow f(x_*) | X, y, x_* \sim \mathcal{N}(m_{post}, k_{post})$

↳ $m_{post} = k_*^T (K + \sigma^2 I)^{-1} y$

↳ $k_{post} = k_{**} - k_*^T (K + \sigma^2 I) k_*$

Algorithm AdaBoost

1) Initialize weights: $w^{[0]}(i) = \frac{1}{n} \quad \forall i$

2) For $m = 1:M$

2.1) Fit $b(l)$ to data to get hard labels $\hat{b}^{[m]}$

2.2) $err^{[m]} = \sum_{i=1}^{n} w^{[m-1](i)} \cdot \mathbb{1}_{\{\hat{y} \neq \hat{b}^{[m]}(x^{(i)})\}}$

2.3) $\hat{\beta}^{[m]} = \frac{1}{2} \log\left(\frac{1 - err^{[m]}}{err^{[m]}}\right)$

2.4) $w^{[m](i)} = w^{[m-1](i)} \cdot \exp\left(\hat{\beta}^{[m]} \cdot y^{(i)} \cdot \hat{b}^{[m]}(x^{(i)})\right)$

2.5) normalize $w^{[m-1](i)}$ to sum 1

3) $\hat{f}(x) = \sum_{m=1}^{M} \hat{\beta}^{[m]} \hat{b}^{[m]}(x)$

Algorithm Forward stagewise add. mod.

1) $\hat{f}^{[0]}(x)$ is optimal const. mod.

2) For $m = 1:M$

2.1) $(\hat{\beta}^{[m]}, \hat{\theta}^{[m]}) = \text{argmin}_{\beta, \theta} \sum_{i=1}^{n} L(y^{(i)}, f^{[m-1]}(x^{(i)}) + \beta b(x^{(i)}, \theta))$

2.2) $\hat{f}^{[m]} = \hat{f}^{[m-1]}(x) + \hat{\beta}^{[m]} b(x, \hat{\theta}^{[m]})$

Algorithm GB

1) Initialize $\hat{f}^{[0]}(x) = \text{argmin}_{\theta_0} \sum_{i=1}^{n} L(y^{(i)}, \theta_0)$

2) For $m = 1:M$

2.1) $\forall i: \tilde{r}^{[m](i)} = -\left[\frac{\delta L(y,f)}{\delta f}\right]_{f = \hat{f}^{[m-1]}(x^{(i)}), y = y^{(i)}}$

2.2) Fit $b(l)$ to $\tilde{r}^{[m]}$: $\hat{\theta}^{[m]} = \text{argmin}_{\theta} \sum_{i=1}^{n} (\tilde{r}^{[m](i)} - b(x^{(i)}, \theta))^2$

2.3) α const small value or optimized

2.4) $\hat{f}^{[m]}(x) = \hat{f}^{[m-1]}(x) + \alpha \cdot b(x, \hat{\theta}^{[m]})$

3) $\hat{f}(x) = \hat{f}^{[M]}(x)$

Algorithm GB Multiclass

1) $\hat{f}_k^{[0]}(x) = 0 \quad , k = 1, \dots, g$

2) For $m = 1:M$

2.1) $\hat{\pi}_k^{[m-1]}(x) = \frac{\exp(\hat{f}_k^{[m-1]}(x))}{\sum_{g} \exp(\hat{f}_g^{[m-1]}(x))} \quad , k = 1, \dots, g$

2.2) For $k = 1:g$

2.2.1) $\forall i: \tilde{r}^{[m](i)} = \mathbb{1}_{\{y^{(i)} = k\}} - \hat{\pi}_k^{[m-1]}(x^{(i)})$

2.2.2) Fit $b_k^{[m]}$ to $\tilde{r}^{[m](i)}$

2.2.3) Update $\hat{f}_k^{[m]} = \hat{f}_k^{[m-1]} + \alpha \, b_k^{[m]}$

3) output $\hat{f}_1^{[m]}, \dots, \hat{f}_g^{[m]}$

Algorithm CWB

1) Initialize $\hat{f}^{[0]}(x) = \text{argmin}_{\theta_0} \sum_{i=1}^{n} L(y^{(i)}, \theta_0)$

2) For $m = 1:M$

2.1) $\forall i: \tilde{r}^{[m](i)} = -\left[\frac{\delta L(y,f)}{\delta f}\right]_{f = \hat{f}^{[m-1]}(x^{(i)}), y = y^{(i)}}$

2.2) for $j = 1:J$

2.2.1) Fit b_j to $\tilde{r}^{[m]}$: $\hat{\theta}_j^{[m]} = \text{argmin}_{\theta} \sum_{i=1}^{n} (\tilde{r}^{[m](i)} - b_j(x^{(i)}, \theta))^2$

2.3) $\hat{b}_j^{[m]} = \text{argmin}_{\hat{\theta}_j^{[m]}} \sum_{i=1}^{n} (\tilde{r}^{[m](i)} - \hat{b}_j(x^{(i)}, \hat{\theta}_j^{[m]}))^2$

2.4) $\hat{f}^{[m]}(x) = \hat{f}^{[m-1]}(x) + \alpha \cdot \hat{b}_j(x, \hat{\theta}^{[m]})$

3) $\hat{f}(x) = \hat{f}^{[M]}(x)$